

1. 题目深度解构

题意重述

有 n 个选手，每个选手有三维坐标 (x_i, y_i, z_i) 和权值 w_i 。

需要把所有选手分成红队和蓝队。要求任意同队两人的三维欧氏距离都至少为 d 。

对每个 $k = 1, 2, \dots, n-1$ ，求所有合法分队方案中，红队人数恰好为 k 时，红队权值乘积之和。

红队权值定义为：

$$\prod_{i \in \text{Red}} w_i$$

答案对 998244353 取模。

约束分析

$$n \leq 10^5, \quad d \leq 10^8$$

暴力枚举所有点对是 $O(n^2)$ ，不可接受。

核心约束是三维几何距离。我们需要判断哪些点对距离小于 d 。这些点对不能在同一队，所以它们必须分到不同队。

因此可以建立一张冲突图：

$$(i, j) \in E \iff \text{dist}(i, j) < d$$

合法分队等价于给冲突图做二染色。每条冲突边两端颜色必须不同。

所以：

如果冲突图不是二分图，所有答案都是 0。

如果是二分图，每个连通块有两种选择：二分图的一侧进红队，另一侧进蓝队；或者反过来。

题目类型判定

这是“几何建图 + 二分图判定 + 生成函数乘法”。

关键难点有两个：

1. 如何避免 $O(n^2)$ 建边；
2. 如何高效合并每个连通块的贡献多项式。

2. 思维路径推导

第一重观察：同队距离限制，其实是冲突图二染色

题目说：

同一队内任意两人的距离都必须至少为 d 。

反过来说，如果两个人距离小于 d ，他们绝对不能在同一队。

因为只有红队和蓝队两个队伍，所以这些冲突点对必须被分到不同队。

于是我们把距离小于 d 的点对连边。问题变成：

每条边两端颜色必须不同。

这就是二分图判定。

如果某个连通块不是二分图，比如出现三角形，那么不可能合法分成两队。

如果某个连通块是二分图，那么它的颜色划分只有两种：原划分或整体翻转。

设某个连通块的两侧分别是 A, B ，那么这个连通块对答案的贡献是：

$$\left(\prod_{i \in A} w_i\right) x^{|A|} + \left(\prod_{i \in B} w_i\right) x^{|B|}$$

所有连通块互相独立，所以总生成函数是这些二项式的乘积。

最后，第 k 项系数就是红队大小为 k 的答案。

第二重观察：冲突图可能很密，但如果还能二分，局部点数一定受限

直接枚举所有点对会炸。

但距离小于 d 是一个几何局部条件。自然想到空间哈希，把三维空间切成边长为 d 的立方体。

两个点如果距离小于 d ，那么它们所在的网格编号在每个维度上最多相差 1。因此每个点只需要检查周围 $3^3 = 27$ 个格子。

问题是：一个格子里可能有很多点，仍然可能退化成 $O(n^2)$ 。

这里有一个关键几何观察：

如果某个边长为 d 的立方体里有超过 16 个点，那么冲突图一定不是二分图。

原因是：把这个立方体再按每个维度对半切，得到 $2^3 = 8$ 个小立方体。每个小立方体的边长是 $d/2$ ，其空间对角线长度为：

$$\sqrt{3} \cdot \frac{d}{2} < d$$

所以同一个小立方体内任意两点距离都小于 d ，它们两两冲突。

如果大立方体内有超过 16 个点，根据抽屉原理，某个小立方体内至少有 3 个点。这 3 个点两两距离小于 d ，形成三角形，冲突图不是二分图。

因此：

如果某个网格内点数 > 16，直接输出全 0。
否则每个点只检查周围 27 个格子，每个格子最多 16 个点。

总检查次数就是 $O(27 \cdot 16 \cdot n)$ ，可以接受。

第三重观察：不用真的存图，用带权并查集维护二分约束

每条冲突边要求：

$$color_u \oplus color_v = 1$$

这是典型的带权并查集。

维护每个点到所在集合根节点的颜色异或值。加入一条边 (u, v) 时，要求它们颜色不同。

如果发现已有关系与新边矛盾，说明图不是二分图，输出全 0。

这样可以边枚举边合并，不需要存所有边。

第四重观察：连通块贡献是二项式，多项式乘起来

对每个最终连通块，统计两侧：

$$cnt_0, cnt_1$$

以及两侧权值乘积：

$$prod_0, prod_1$$

该连通块贡献：

$$prod_0 x^{cnt_0} + prod_1 x^{cnt_1}$$

所有连通块贡献相乘。

由于 $n \leq 10^5$ ，需要用 NTT 分治合并多项式。

可以进一步把每个二项式提取较小指数：

若 cnt_0 ，则：

$$prod_0 x^{cnt_0} + prod_1 x^{cnt_1} = x^{cnt_0} (prod_0 + prod_1 x^{cnt_1 - cnt_0})$$

把所有提出来的 $x^{\min(cnt_0, cnt_1)}$ 累计成一个整体偏移 `shift`，只乘剩下的稀疏二项式。

这样多项式总次数不会超过 n 。

3. Trick 与陷阱

本题核心 Trick 有三个。

第一，距离限制冲突图。距离小于 d 的点对必须异色，所以是二分图问题。

第二，三维网格剪枝。边长为 d 的格子只需要检查周围 27 个格子；如果单格点数超过 16，必有三角形，直接判无解。

第三，二分图连通块转生成函数。每个连通块只有“选左侧进红”或“选右侧进红”两种选择，所以贡献是一个二项式。

易错点：

1. 距离条件是小于 d 才冲突，不是小于等于 d 。
2. 比较距离时必须用平方，避免浮点误差。
3. 坐标和 d^2 需要 `long long`。
4. 每个格子最多允许 16 个点；超过 16 可以直接判无解。
5. 输出只需要 $k = 1$ 到 $n - 1$ ，不用输出 $k = 0$ 和 $k = n$ 。

4. 代码实现 (C++23, jiangly 风格)

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

constexpr int P = 998244353;
constexpr int G = 3;

int power(int a, i64 b) {
    int r = 1;
    while (b) {
        if (b & 1) {
            r = 1LL * r * a % P;
        }
        a = 1LL * a * a % P;
        b >>= 1;
    }
    return r;
}

void ntt(vector<int> &a, bool inv) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
```

```

        swap(a[i], a[j]);
    }
}

for (int len = 2; len <= n; len <= 1) {
    int wlen = power(G, (P - 1) / len);
    if (inv) {
        wlen = power(wlen, P - 2);
    }

    for (int i = 0; i < n; i += len) {
        int w = 1;
        for (int j = 0; j < len / 2; j++) {
            int u = a[i + j];
            int v = 1LL * a[i + j + len / 2] * w % P;

            a[i + j] = u + v;
            if (a[i + j] >= P) {
                a[i + j] -= P;
            }

            a[i + j + len / 2] = u - v;
            if (a[i + j + len / 2] < 0) {
                a[i + j + len / 2] += P;
            }

            w = 1LL * w * wlen % P;
        }
    }
}

if (inv) {
    int iv = power(n, P - 2);
    for (auto &x : a) {
        x = 1LL * x * iv % P;
    }
}

}

vector<int> mul(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) {
        return {};
    }

    int need = a.size() + b.size() - 1;

    if (min(a.size(), b.size()) <= 8 || 1LL * a.size() * b.size() <= 250000) {
        vector<int> c(need);
        for (int i = 0; i < int(a.size()); i++) {
            if (!a[i]) {
                continue;
            }
            for (int j = 0; j < int(b.size()); j++) {

```

```

        c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % P;
    }
}
return c;
}

int n = 1;
while (n < need) {
    n <= 1;
}

a.resize(n);
b.resize(n);

ntt(a, false);
ntt(b, false);

for (int i = 0; i < n; i++) {
    a[i] = 1LL * a[i] * b[i] % P;
}

ntt(a, true);
a.resize(need);

return a;
}

struct DSU {
    vector<int> p, sz, d;

    DSU(int n) : p(n), sz(n, 1), d(n) {
        iota(p.begin(), p.end(), 0);
    }

    int find(int x) {
        if (p[x] == x) {
            return x;
        }
        int y = p[x];
        p[x] = find(y);
        d[x] ^= d[y];
        return p[x];
    }

    bool merge(int a, int b, int w) {
        int ra = find(a);
        int rb = find(b);
        int da = d[a];
        int db = d[b];

        if (ra == rb) {
            return (da ^ db) == w;
        }
    }
}

```

```

        if (sz[ra] < sz[rb]) {
            swap(ra, rb);
            swap(da, db);
        }

        p[rb] = ra;
        d[rb] = da ^ db ^ w;
        sz[ra] += sz[rb];

        return true;
    }
};

struct Cell {
    i64 x, y, z;

    bool operator==(const Cell &o) const {
        return x == o.x && y == o.y && z == o.z;
    }
};

struct Hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(const Cell &c) const {
        uint64_t h = splitmix64(c.x);
        h ^= splitmix64(c.y + 0x9e3779b97f4a7c15);
        h ^= splitmix64(c.z + 0xbf58476d1ce4e5b9);
        return h;
    }
};

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int n;
    i64 d;
    cin >> n >> d;

    vector<i64> x(n), y(n), z(n);
    vector<int> w(n);
    vector<Cell> cell(n);

    unordered_map<Cell, vector<int>, Hash> mp;
    mp.reserve(n * 2);

    bool bad = false;

```

```

for (int i = 0; i < n; i++) {
    cin >> x[i] >> y[i] >> z[i] >> w[i];

    cell[i] = {x[i] / d, y[i] / d, z[i] / d};

    auto &v = mp[cell[i]];
    v.push_back(i);

    if (int(v.size()) > 16) {
        bad = true;
    }
}

auto output_zero = [&]() {
    for (int i = 1; i < n; i++) {
        cout << 0 << '\n';
    }
};

if (bad) {
    output_zero();
    return 0;
}

DSU dsu(n);

i64 dd = d * d;

for (int i = 0; i < n && !bad; i++) {
    for (int dx = -1; dx <= 1 && !bad; dx++) {
        for (int dy = -1; dy <= 1 && !bad; dy++) {
            for (int dz = -1; dz <= 1 && !bad; dz++) {
                cell c {
                    cell[i].x + dx,
                    cell[i].y + dy,
                    cell[i].z + dz
                };

                auto it = mp.find(c);
                if (it == mp.end()) {
                    continue;
                }

                for (auto j : it->second) {
                    if (j <= i) {
                        continue;
                    }

                    i64 a = x[i] - x[j];
                    i64 b = y[i] - y[j];
                    i64 c = z[i] - z[j];

```



```

        if (a * a + b * b + c * c < dd) {
            if (!dsu.merge(i, j, 1)) {
                bad = true;
                break;
            }
        }
    }
}

if (bad) {
    output_zero();
    return 0;
}

vector<array<int, 2>> cnt(n);
vector<array<int, 2>> prod(n);

for (int i = 0; i < n; i++) {
    prod[i] = {1, 1};
}

for (int i = 0; i < n; i++) {
    int r = dsu.find(i);
    int c = dsu.d[i];

    cnt[r][c]++;
    prod[r][c] = 1LL * prod[r][c] * w[i] % P;
}

int coef = 1;
int shift = 0;
vector<vector<int>> polys;

for (int i = 0; i < n; i++) {
    if (dsu.find(i) != i) {
        continue;
    }

    int c0 = cnt[i][0];
    int c1 = cnt[i][1];
    int p0 = prod[i][0];
    int p1 = prod[i][1];

    if (c0 == c1) {
        shift += c0;
        coef = 1LL * coef * ((p0 + p1) % P) % P;
    } else if (c0 < c1) {
        shift += c0;
        vector<int> f(c1 - c0 + 1);
        f[0] = p0;
    }
}

```

```

        f[c1 - c0] = p1;
        polys.push_back(move(f));
    } else {
        shift += c1;
        vector<int> f(c0 - c1 + 1);
        f[0] = p1;
        f[c0 - c1] = p0;
        polys.push_back(move(f));
    }
}

if (coef == 0) {
    output_zero();
    return 0;
}

vector<int> f;

if (polys.empty()) {
    f = {1};
} else {
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;

    for (int i = 0; i < int(polys.size()); i++) {
        pq.push({int(polys[i].size()), i});
    }

    while (pq.size() > 1) {
        auto [_, i] = pq.top();
        pq.pop();

        auto [__, j] = pq.top();
        pq.pop();

        auto a = move(polys[i]);
        auto b = move(polys[j]);

        polys.push_back(mul(move(a), move(b)));
        pq.push({int(polys.back().size()), int(polys.size()) - 1});
    }

    f = move(polys[pq.top().second]);
}

for (int k = 1; k < n; k++) {
    int t = k - shift;
    int ans = 0;

    if (0 <= t && t < int(f.size())) {
        ans = 1LL * coef * f[t] % P;
    }

    cout << ans << '\n';
}

```

```
}  
  
    return 0;  
}
```

5. 总结与启示

预估难度：Codeforces 2400 左右。

核心启示：

当“两队内都不能出现某类点对”时，把这类点对建成冲突边，问题通常会变成二分图染色；每个二分连通块的两
种翻转选择可以自然写成生成函数因子。

这题最值得记住的是几何剪枝：边长为 d 的网格配合“单格超过 16 必有三角形”，可以把看似稠密的三维距离图压到线性级别处理。

